

「2025년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2025.04.20

7 주차



정렬

◆ 목차

06-3 단순 선택 정렬

06-4 단순 삽입 정렬

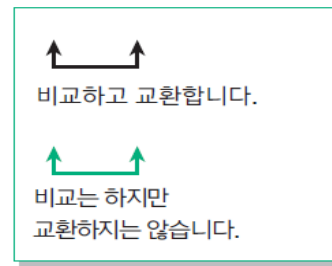
06-5 셀 정렬



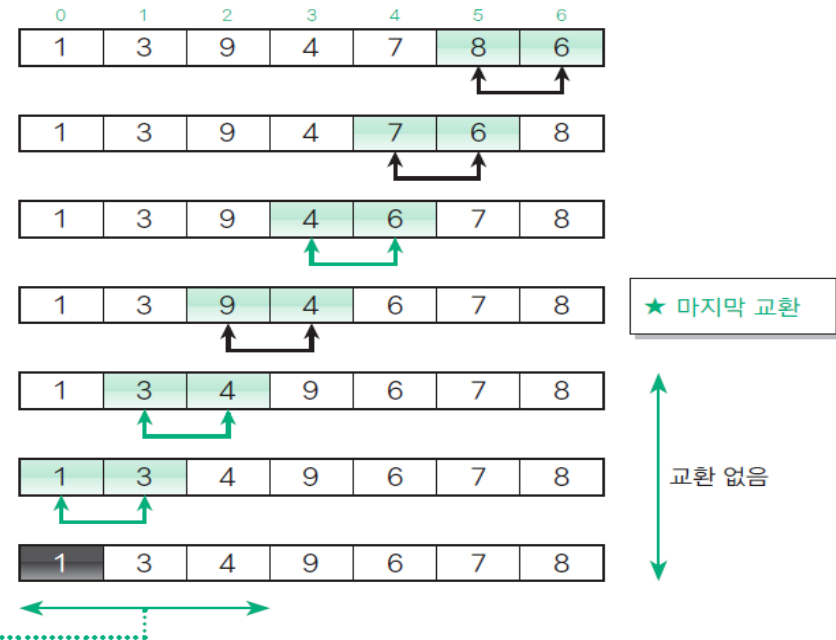
06-2 버블 정렬

◆알고리즘 개선 - (2)

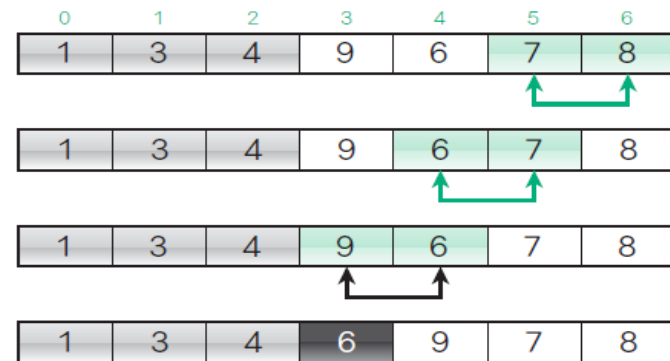
- 마지막 교환(★) 이후에 앞쪽의 세 요소({1, 3, 4})는 정렬된 상태
- 어떤 시점 이후에 교환이 수행되지 않는다면 그보다 앞쪽의 요소는 이미 정렬을 마친 상태라고 생각해도 좋음



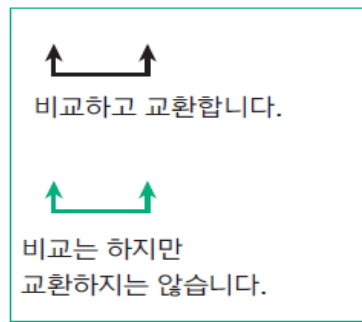
마지막으로 교환을 한 위치 이후의 요소는 정렬을 마친 상태입니다.



[그림 6-8] 버블 정렬의 첫 번째 패스



[그림 6-9] 버블 정렬의 두 번째 패스





06-2 버블 정렬

◆ 실습 6-3

- 버블 정렬(버전 3)
- 교환을 수행할 때마다 오른쪽 요소의 인덱스값을 last에 저장하므로, 패스를 마친 시점에 last에는 마지막으로 교환한 두 요소 가운데 오른쪽 요소의 인덱스가 저장됨
- 이 last값을 k에 대입하여 다음에 수행할 패스의 범위를 a[k]까지로 제한
- 다음 패스에서 마지막으로 비교할 두 요소는 a[k]와 a[k + 1]가 됨
- bubble 함수의 시작 부분에서 k 값을 0으로 초기화하는 이유는 첫 번째 패스에서는 모든 요소를 검사해야 하기 때문

Chapt06_실습 6-3/bubble3.c

```
// 버블 정렬(버전 3: 스캔 범위를 제한)
#include <stdio.h>
#include <stdlib.h>
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)

/*--- 버블 정렬(버전 3: 스캔 범위를 제한) ---*/
void bubble(int a[], int n)
{
    int k = 0; // a[k]보다 앞쪽의 요소는 정렬을 마친 상태

    while (k < n - 1) {
        int last = n - 1; // 마지막으로 교환을 수행한 위치를 저장
        for (int j = n - 1; j > k; j--)
            if (a[j - 1] > a[j]) {
                swap(int, a[j - 1], a[j]);
                last = j; // 마지막으로 교환을 수행한 위치를 저장
            }
        k = last;
    }
}
```



06-3 단순 선택 정렬

◆ 단순 선택 정렬 straight selection sort

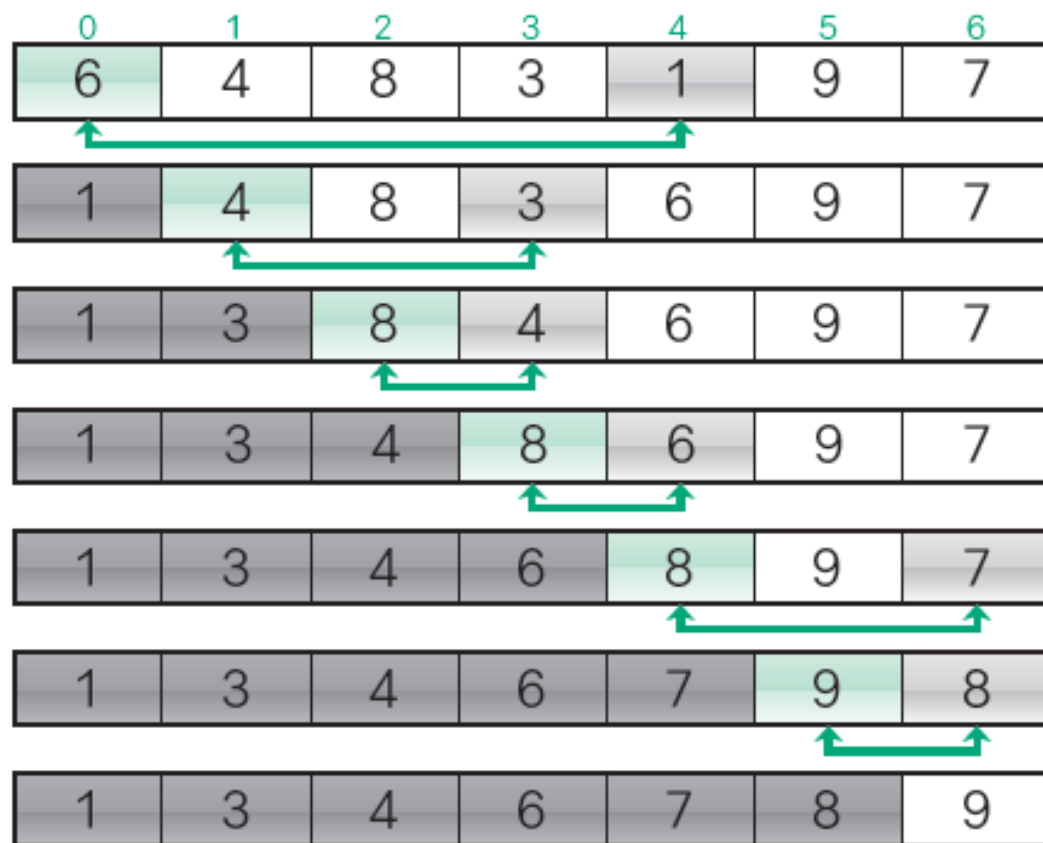
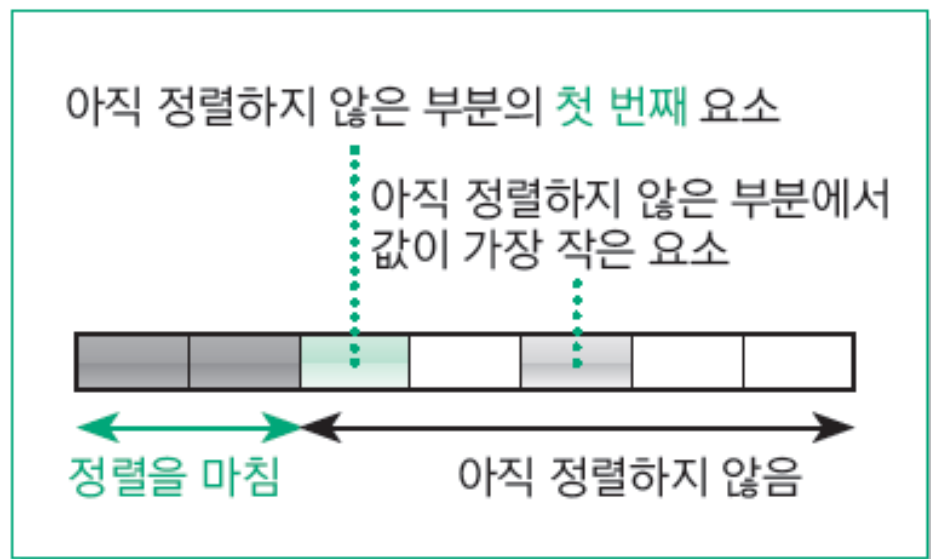
- 아직 정렬하지 않은 부분에서 값이 가장 작은 요소를 선택하고 아직 정렬하지 않은 부분의 첫 번째 요소와 교환

◆ 교환 과정

- 1. 아직 정렬하지 않은 부분에서 가장 작은 키의 값($a[\min]$)을 선택
- 2. $a[\min]$ 과 아직 정렬하지 않은 부분의 첫 번째 요소를 교환



06-3 단순 선택 정렬



[그림 6-10] 단순 선택 정렬 과정



06-3 단순 선택 정렬

◆ 실습 6-4

- 단순 선택 정렬을 수행하는 함수
- 단순 선택 정렬 알고리즘의 요솟값을 비교하는 횟수는 $(n^2 - n) / 2$ 회
- 이 정렬 알고리즘은 서로 떨어져 있는 요소를 교환하는 것이기 때문에 안정적이지 않음

Chapt06_실습 6-4/selection.c

// 단순 선택 정렬

```
#include <stdio.h>
```

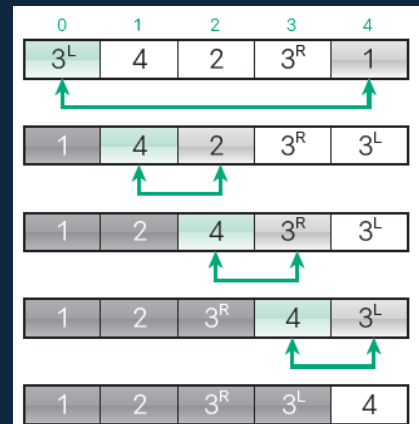
```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)
```

/*--- 단순 선택 정렬 ---*/

```
void selection(int a[], int n)
```

```
{  
    for (int i = 0; i < n - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < n; j++)  
            if (a[j] < a[min])  
                min = j;  
        swap(int, a[i], a[min]);  
    }  
}
```



[그림 6-11] 3^L과 3^R의 위치가 뒤바뀌는 과정



06-4 단순 삽입 정렬

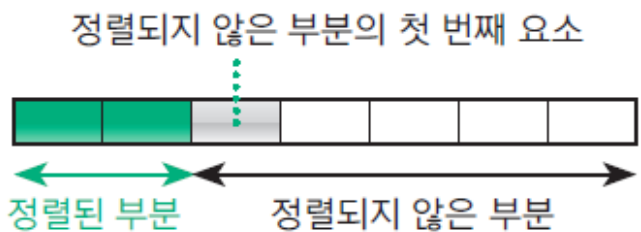
◆ 단순 삽입 정렬 straight insertion sort

- 선택한 요소를 그보다 더 앞쪽의 알맞은 위치에 '삽입하는' 작업을 반복하여 정렬하는 알고리즘
- 아직 정렬되지 않은 부분의 첫 번째 요소를 정렬된 부분의 알맞은 위치에 삽입
- 아래의 두 조건 중 하나를 만족할 때까지 j 를 1씩 감소하면서 대입하는 작업을 반복
 - 1. 정렬된 열의 왼쪽 끝에 도달
 - 2. tmp 보다 작거나 같은 key 를 갖는 항목 $a[j - 1]$ 을 발견
- 드모르간 법칙 적용을 적용해 아래의 두 조건이 모두 성립할 때까지 반복
 - 1. j 가 0보다 큼
 - 2. $a[j - 1]$ 값이 tmp 보다 큼



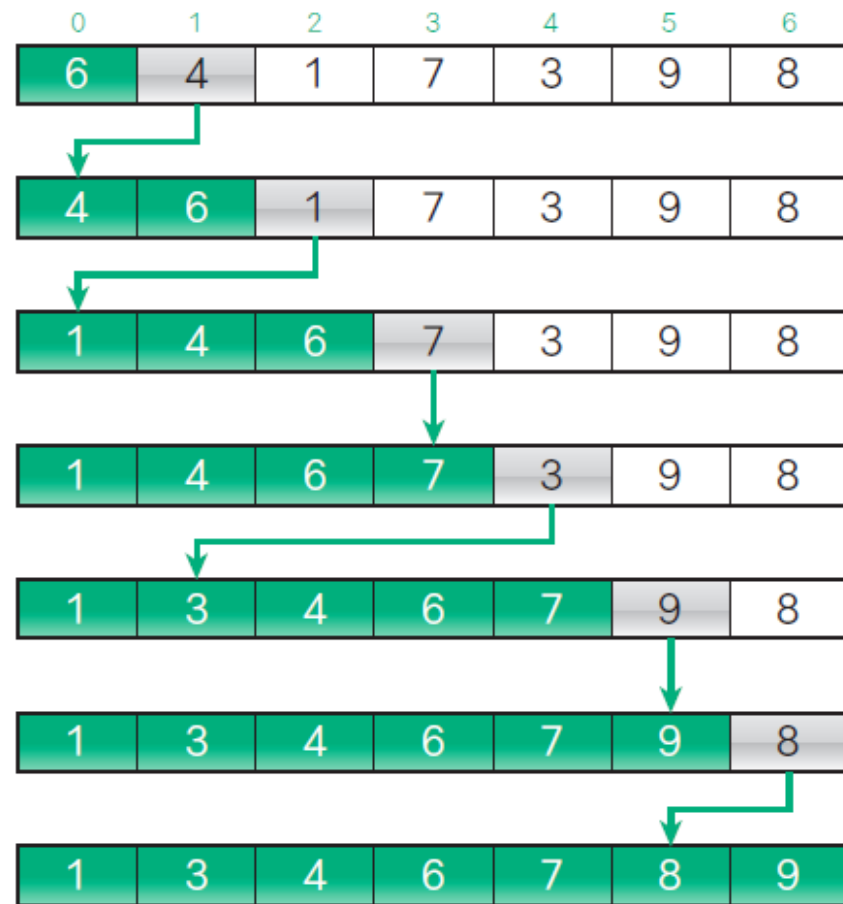
06-4 단순 삽입 정렬

정렬되지 않은 부분의 첫 번째 요소를 정렬된 열의 '알맞은 위치에 삽입'하는 작업을 $n-1$ 회 반복합니다.



정렬된 부분 $a[0], a[1], \dots, a[i - 1]$

정렬되지 않은 부분 $a[i], a[i + 1], \dots, a[n - 1]$



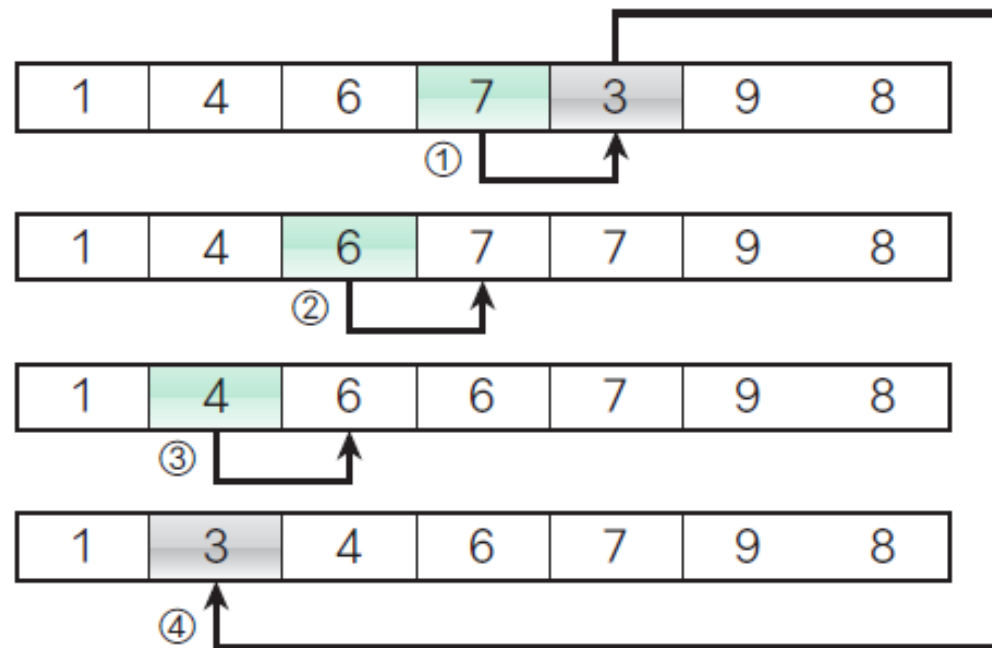
[그림 6-12] 단순 삽입 정렬 과정



06-4 단순 삽입 정렬

- ①~③ ... 3보다 작은 요소를 만날 때까지
이웃한 왼쪽의 요소를 대입하는
작업을 반복합니다.
- ④ ... 멈춘 위치에 3을 대입합니다.

```
j = i;  
tmp = a[i];  
while (j > 0 && a[j - 1] > tmp)  
    a[j] = a[j - 1];  
    j--;  
a[j] = tmp;
```



[그림 6-13] 단순 삽입 정렬에서 요소 3의 삽입 과정



06-4 단순 삽입 정렬

◆ 실습 6-5 - (1)

- 단순 삽입 정렬 알고리즘은 떨어져 있는 요소들이 서로 뒤바뀌지 않아 안정적
- 요소의 비교 횟수와 교환 횟수는 $n^2 / 2$ 회

Chapt06_실습 6-5/insertion.c

// 단순 삽입 정렬

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)
```

/*--- 단순 삽입 정렬 ---*/

```
void insertion(int a[], int n)
```

```
{
```

```
    for (int i = 1; i < n; i++) {
```

```
        int tmp = a[i];
```

```
        int j;
```

```
        for (j = i; j > 0 && a[j - 1] > tmp; j--)
```

```
            a[j] = a[j - 1];
```

```
            a[j] = tmp;
```

```
    }
```

```
}
```

실행 결과

단순 삽입 정렬

요소 개수: 7

x[0]: 6

x[1]: 4

x[2]: 3

x[3]: 7

x[4]: 1

x[5]: 9

x[6]: 8

오름차순으로 정렬했습니다.

x[0] = 1

x[1] = 3

x[2] = 4

x[3] = 6

x[4] = 7

x[5] = 8

x[6] = 9



06-4 단순 삽입 정렬

◆ 실습 6-5 - (2)

- 단순 삽입 정렬 프로그램

◆ 단순 정렬의 시간 복잡도

- 지금까지 공부한 세 가지 단순 정렬 (버블, 선택, 삽입)의 시간 복잡도는 모두 $O(n^2)$ 으로 효율이 좋지 않음

```
int main(void)
{
    int nx;

    puts("단순 삽입 정렬");
    printf("요소 개수: ");
    scanf("%d", &nx);
    int* x = calloc(nx, sizeof(int)); // 요소의 개수가 nx인 int형 배열 x를 생성

    for (int i = 0; i < nx; i++) {
        printf("x[%d] : ", i);
        scanf("%d", &x[i]);
    }

    insertion(x, nx); // 배열 x를 단순 삽입 정렬

    puts("오름차순으로 정렬했습니다.");
    for (int i = 0; i < nx; i++)
        printf("x[%d] = %d\n", i, x[i]);

    free(x); // 배열 해제

    return 0;
}
```